

LAPORAN IMPLEMENTASI: Persistensi Data Menggunakan SQLite di Flutter

Nama : Zenny Nurul Zulaikoh
NIM : 43050240056
Kelas : Teknologi Informasi – 4C
Link Github : https://github.com/zennynurul/belajar_database_sql

1. Pendahuluan

Pada tahap pengembangan sebelumnya, saya telah berhasil membangun antarmuka aplikasi User List berserta fitur CRUD dasar. Namun, penyimpanan data masih mengandalkan memori lokal statis (`List<UserModel>`). Hal ini menyebabkan data bersifat *temporary* (sementara); setiap kali aplikasi di- *restart* atau kode di- *reload*, seluruh data pengguna hilang. Untuk menyelesaikan masalah ini, saya melakukan migrasi dari penyimpanan memori statis ke basis data fisik menggunakan SQLite. Implementasi ini memastikan persistensi data, sehingga aplikasi dapat menyimpan, memperbarui, membaca, dan menghapus data secara permanen di dalam ruang penyimpanan perangkat keras (HP).

2. Persiapan Dependensi (*Dependencies*)

Langkah pertama yang saya lakukan adalah menambahkan pustaka (*library*) yang dibutuhkan ke dalam berkas `pubspec.yaml` melalui terminal:

- **sqlite**: Pustaka resmi untuk menjalankan *query* SQLite di Flutter.
- **path**: Digunakan untuk memetakan dan menggabungkan lokasi direktori penyimpanan basis data di dalam sistem Android/iOS.

3. Kode Implementasi Lengkap

Seluruh komponen mulai dari Model, Database Helper, hingga User Interface disatukan dalam blok kode di bawah ini :

```
import 'package:flutter/material.dart';
import 'package:sqlite/sqlite.dart';
import 'package:path/path.dart' as p;

void main() {
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(home: ListUserDataPage());
  }
}
```

```

    }
}

class DatabaseHelper {
    static Database? _database;

    static Future<Database> get database async {
        if (_database != null) return _database!;
        _database = await _initDB();
        return _database!;
    }

    static Future<Database?> _initDB() async {
        String path = p.join(await getDatabasesPath(), 'user_db.db');
        return await openDatabase(
            path,
            version: 1,
            onCreate: (db, version) {
                return db.execute(
                    'CREATE TABLE users(id INTEGER PRIMARY KEY AUTOINCREMENT, nama TEXT, umur
INTEGER)',
                );
            },
        );
    }

    //create
    static Future<int> insertUser(UserModel userModel) async {
        final db = await database;
        Map<String, dynamic> user = userModel.toJson();
        return await db.insert(
            'users',
            user,
            conflictAlgorithm: ConflictAlgorithm.replace,
        );
    }

    //read

    static Future<List<UserModel>> getData() async {
        final db = await database;
        List<Map<String, Object?>> result = await db.query('users');

        List<UserModel> users = result.map((userMap) {
            return UserModel.fromJson(userMap);
        }).toList();
    }
}

```

```

    return users;
}

//update

static Future<int> updateData(int id, UserModel userModel) async {
    final db = await database;
    var user = userModel.toJson()..remove("id");
    return await db.update("users", user, where: "id = ?", whereArgs: [id]);
}

//delete
static Future<int> deleteData(int id) async {
    final db = await database;
    return await db.delete("users", where: "id = ?", whereArgs: [id]);
}
}

class ListUserDataPage extends StatefulWidget {
    const ListUserDataPage({super.key});

    @override
    State<ListUserDataPage> createState() => _ListUserDataPageState();
}

class UserModel {
    int? id;
    String nama = "";
    int umur = 0;

    UserModel({this.id, required this.nama, required this.umur});

    //convert dari map / hashmap ke model
    factory UserModel.fromJson(Map<String, dynamic> json) {
        return UserModel(id: json['id'], nama: json['nama'], umur: json['umur']);
    }

    // convert dari model ke map

    Map<String, dynamic> toJson() {
        return {'id': id, 'nama': nama, 'umur': umur};
    }
}

class _ListUserDataPageState extends State<ListUserDataPage> {
    final TextEditingController _namaCtrl = TextEditingController();
    final TextEditingController _umurCtrl = TextEditingController();

```

```

List<UserModel> userList = [];

@override
void initState() {
  super.initState();

  _reloadData();
}

void _reloadData() async {
  var users = await DatabaseHelper.getData();

  setState(() {
    userList = users;
  });
}

void _form(int? id) {
  if (id != null) {
    var user = userList.firstWhere((data) => data.id == id);
    _namaCtrl.text = user.nama;
    _umurCtrl.text = user.umur.toString();
  } else {
    _namaCtrl.text = "";
    _umurCtrl.text = "";
  }
}

showModalBottomSheet(
  context: context,
  isScrollControlled: true,
  builder: (context) => Padding(
    padding: EdgeInsetsGeometry.fromLTRB(
      20,
      20,
      20,
      MediaQuery.of(context).viewInsets.bottom + 50,
    ),
    child: Column(
      mainAxisAlignment: MainAxisAlignment.min,
      children: [
        TextField(
          controller: _namaCtrl,
          decoration: InputDecoration(hintText: "Nama"),
        ),
        TextField(
          controller: _umurCtrl,

```

```

        decoration: InputDecoration(hintText: "Umur"),
        keyboardType: TextInputType.number,
    ),
    ElevatedButton(
        onPressed: () =>
            _save(id, _namaCtrl.text, int.parse(_umurCtrl.text)),
        child: Text(id == null ? "Tambah" : "Perbaharui"),
    ),
],
),
),
);
}

void _save(int? id, String nama, int umur) async {
    var newUser = UserModel(nama: nama, umur: umur);
    if (id != null) {
        await DatabaseHelper.updateData(id, newUser);
    } else {
        await DatabaseHelper.insertUser(newUser);
    }
    _reloadData();
    Navigator.pop(context);
}

void _delete(int id) {
    showDialog(
        context: context,
        builder: (context) => AlertDialog(
            title: Text("Konfirmasi Hapus"),
            content: Text("Apakah anda yakin ingin menghapus data ini?"),
            actions: [
                TextButton(
                    onPressed: () {
                        Navigator.pop(context);
                    },
                    child: Text("Batal"),
                ),
                TextButton(
                    onPressed: () async {
                        await DatabaseHelper.deleteData(id);
                        _reloadData();
                        Navigator.pop(context);
                    },
                    child: Text("Hapus"),
                ),
            ],
        ),
    );
}

```

```

    ),
  );
}

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(title: Text("User List")),
    body: ListView.builder(
      itemCount: userList.length,
      itemBuilder: (cxt, i) => ListTile(
        title: Text(userList[i].nama),
        subtitle: Text("umur : ${userList[i].umur} tahun"),
        trailing: Row(
          mainAxisAlignment: MainAxisAlignment.min,
          children: [
            TextButton(
              onPressed: () => _form(userList[i].id),
              child: Icon(Icons.edit),
            ),
            TextButton(
              onPressed: () => _delete(userList[i].id!),
              child: Icon(Icons.delete),
            ),
          ],
        ),
      ),
    floatingActionButton: FloatingActionButton(
      onPressed: () => _form(null),
      child: Icon(Icons.add),
    ),
  );
}
}

```

4. Penjelasan

A. Mekanisme Komunikasi Data (Mapping)

Karena SQLite adalah basis data relasional, ia hanya menyimpan data dalam bentuk tabel (*Map*). Saya mengimplementasikan pola Serialization pada UserModel:

- **fromJson:** Mengubah baris tabel dari SQLite menjadi objek Dart agar bisa ditampilkan di ListTile.
- **toJson:** Mengubah data dari formulir input menjadi format baris tabel agar bisa diterima oleh perintah db.insert.

B. Manajemen Koneksi (Database Helper)

Saya menggunakan pola Singleton pada kelas DatabaseHelper. Hal ini memastikan bahwa aplikasi hanya membuka satu jalur koneksi ke fail `user_db.db`. Penggunaan AUTOINCREMENT pada tabel membebaskan saya dari kewajiban mengelola ID secara manual, karena SQLite secara otomatis memberikan ID unik pada setiap data baru.

C. Pengamanan Logika UI & Memori

Saya menyadari bahwa performa aplikasi tidak hanya bergantung pada *database*, tetapi juga efisiensi kode UI:

1. **dispose():** Saya menyertakan fungsi ini untuk memastikan TextEditingController dimatikan saat halaman tidak digunakan, guna menghindari kebocoran memori.
2. **int.tryParse():** Ini adalah baris pertahanan kritis. Jika pengguna salah memasukkan input (misal huruf pada kolom umur), fungsi ini mengembalikan nilai null alih-alih menyebabkan aplikasi *crash*.
3. **_reloadData():** Setiap kali ada operasi tulis (Simpan/Hapus), saya memicu pemanggilan ulang data dari basis data untuk memastikan tampilan UI selalu sinkron dengan kondisi penyimpanan fisik terbaru.

5. Kesimpulan

Implementasi ini berhasil memindahkan seluruh alur kerja aplikasi ke sistem penyimpanan permanen. Dengan struktur kode satu halaman ini, alur data dari *input user* -> *konversi model* -> *database helper* -> *tampilan list* menjadi lebih mudah dipahami dan dikelola. Laporan ini membuktikan bahwa aplikasi kini telah siap digunakan secara nyata tanpa risiko kehilangan data saat aplikasi ditutup.